

DineFlow — Restaurant Ordering & Management Platform

A complete restaurant SaaS built from scratch: a customer ordering storefront and an admin management dashboard, architected to grow from a mock-data prototype into a live, database-backed product.

[View Live Site →](#)[View Source \(GitHub\)](#)

ROLE

TIMELINE

STACK

STATUS

Full-Stack Developer Solo project **Next.js · TypeScript · Supabase** Live — in active development

OVERVIEW

What DineFlow is

DineFlow is a two-sided restaurant platform: a polished customer storefront for browsing a menu, ordering food, and tracking delivery status, paired with a full admin dashboard for managing the menu, orders, payments, and sales analytics — the kind of internal tooling a real restaurant business runs on day to day.

The project was built in two deliberate stages: first a complete, realistic frontend using mock data to validate every user flow and get the design right, then a systematic migration to a real PostgreSQL database, authentication, and cloud deployment — mirroring how a lot of real product work actually happens.

Customer Experience

Menu browsing with search & filters, cart, checkout, live order tracking, and order history.

Admin Dashboard

Food & category management, order pipeline control, payment overview, and sales analytics.

Production Path

Architected from day one to swap mock data for a real database without a rewrite.

TECH STACK

Built with a modern, production-grade toolset

Every choice was made with a real deployment in mind, not just a demo.

Next.js 14 (App Router)

TypeScript

Tailwind CSS

Prisma ORM

PostgreSQL (Supabase)

Supabase Storage

NextAuth.js

Recharts

Vercel

KEY FEATURES

What's fully working

- ✓ Menu browsing with live search, category filters, and price filters
- ✓ Persistent shopping cart with quantity controls
- ✓ Full checkout flow — delivery/pickup, payment method selection
- ✓ Visual, real-time order status tracker (Placed → Confirmed → Preparing → Ready → Completed)
- ✓ Admin CRUD for foods and categories, including availability toggles
- ✓ Admin order pipeline — filter by status, update status, reflected instantly on the customer side
- ✓ Dashboard analytics — revenue & order trend charts, category performance, top sellers
- ✓ Fully responsive across mobile, tablet, and desktop

SCREENSHOTS

Product walkthrough

Replace these placeholders with real screenshots once the UI is finalized.

Homepage

Hero, featured categories, popular dishes

Menu & Search

Filtering, sorting, category tabs

Order Tracking

Visual status pipeline

Admin Dashboard

Revenue charts & recent orders

Challenges & how they were solved

DATA LAYER

Designing for a mock-to-real migration from day one

Rather than hardcoding data into components, all catalog and order state was routed through dedicated React Context providers (CatalogContext, OrderContext) with a shape matching the eventual Prisma schema. This meant the entire admin CRUD experience could be built and fully tested before a database even existed — and migrating to real API calls later is a contained change, not a rewrite.

DATABASE

Serverless-safe Postgres connections

Supabase's connection pooler (PgBouncer) doesn't support prepared statements the way some tooling expects. The project uses two distinct connection strings — a pooled connection for the running app (safe under Vercel's serverless concurrency) and a direct connection reserved for Prisma CLI migrations — avoiding a class of connection-exhaustion bugs common in serverless deployments.

CONTENT INTEGRITY

Verifying every image against a live source

Placeholder imagery sourced from memory included several invalid CDN references. Rather than leave broken images in a "finished" product, every food and category image was individually verified against Unsplash's live search index and replaced with confirmed-working URLs — a small detail, but the kind that separates a demo from something a client can actually show customers.

PROJECT STATUS

Where things stand

AREA	STATUS
Frontend UI (customer + admin)	Complete
Live deployment (Vercel)	Complete
Database schema & hosting (Supabase)	Complete
API layer connecting UI to database	In Progress
Authentication & protected admin routes	Planned
Live payment gateway integration	Planned
Image uploads via Supabase Storage	Planned

Roadmap to full production

The remaining work follows a deliberate sequence: wire the admin dashboard and storefront to real API routes backed by the Supabase database, add NextAuth-based authentication to lock down the admin panel and support real customer accounts, then layer in a live payment gateway suited to the target market, image uploads through Supabase Storage, and transactional email notifications — each phase tested in isolation before moving to the next.

Interested in the full build or working together?

[Live Demo](#) · [GitHub Repository](#) · [LinkedIn](#)